Na'vi Language Translator

End-to-End MLOps Application

DA6360 — MLOps Course Project
Repository: github.com/usnaveen/navi-translator
Date: 28 April 2026

Submission Items	
GitHub Repository	https://github.com/usnaveen/navi-translator
Architecture Document	docs/HLD.md
API Specification	docs/LLD.md
Test Plan & Report	docs/TEST_PLAN.md / TEST_REPORT.md
User Manual	docs/user_manual.md
Automated Tests	32 / 32 passing (pytest)

1. Executive Summary

Na'vi Translator is a production-grade MLOps application that translates Na'vi — a low-resource constructed language from the *Avatar* film universe — into English from both audio and text input. The system is built end-to-end following the full MLOps lifecycle: data ingestion → versioning → training → experiment tracking → packaging → deployment → monitoring → automated retraining.

The core engineering challenge is **low-resource adaptation under a strict no-cloud constraint**. Na'vi has approximately 2,600 documented word pairs and limited public audio data. All training runs on local Apple Silicon hardware using PEFT/LoRA to make Whisper fine-tuning feasible without GPU cloud resources.

Key Achievements

- Full MLOps pipeline implemented: DVC data versioning, Airflow orchestration, MLflow experiment tracking, Prometheus/Grafana monitoring
- Two trained models: Whisper-tiny + LoRA (ASR) and MarianMT (NMT), both registered in MLflow Model Registry
- FastAPI backend with 6 REST endpoints, nginx frontend, Docker Compose with 5 isolated services
- 32 / 32 automated tests passing (unit + integration); CI/CD via GitHub Actions
- Reykunyu dictionary fallback for low-confidence outputs — keeps UX usable when neural confidence is below threshold

2. Problem Statement & Success Metrics

Na'vi has an active hobbyist learning community but no unified translation tool. Static dictionary lookups exist but cannot handle audio, multi-word phrases, or community vocabulary contributions. The goal is a single system handling speech, text, and community submissions with full MLOps observability.

Type	Metric	Target	Result
ML	Whisper ASR — Word Error Rate	≤ 0.35	0.42 (low-resource constrained)
ML	MarianMT BLEU score	≥ 0.25	Logged in MLflow registry
Business	Text translation latency	< 200 ms	~85 ms (dictionary path)
Business	Audio translation latency	< 5 s	~1.2 s (whisper-tiny)
Business	OOV drift alert threshold	< 15%	Prometheus alert configured

3. System Architecture

The system is split into four layers — Data, Training, Serving, and Monitoring — each independently containerised and connected only through defined interfaces.

Layer	Components	Tools

Data	Ingestion, versioning, preprocessing, splitting, baseline stats	DVC, Reykunyu API, gTTS, Airflow
Training	Whisper-tiny + LoRA (ASR), MarianMT (NMT), evaluation, auto-promotion	MLflow, PEFT, HuggingFace Transformers
Serving	REST API, model inference, fallback dictionary, metrics export	FastAPI, nginx, Prometheus client
Monitoring	Metric scraping, dashboards, drift detection, retrain alerts	Prometheus, Grafana, EvidentlyAI

Request Flow — Audio Translation

```
Browser MediaRecorder → POST /api/translate/audio → Resample 16 kHz mono → Whisper-tiny encoder → LoRA-adapted decoder → Na'vi text → MarianMT → English → if confidence < 0.4: Reykunyu word-by-word fallback → JSON response
```

Docker Compose Network

```
navi-net (bridge) ■■■ backend :8000 FastAPI inference server ■■■ frontend :3000 nginx static UI ■■■ mlflow :5000 experiment tracking server ■■■ prometheus :9090 metric scraper ■■■ grafana :3001 NRT dashboard
```

4. MLOps Implementation

4.1 Data Engineering

The data pipeline is defined as a DVC DAG in **dvc.yaml** with five stages. Each stage declares its source-code dependencies, parameter dependencies (from params.yaml), and output artifacts so DVC can detect changes and re-run only what is necessary. Apache Airflow orchestrates the same pipeline in production with task-level retry and DAG-level scheduling.

Stage	Input	Output	Tool
ingest	Reykunyu API + TTS	data/raw/words.json + audio	requests, gTTS
preprocess_text	words.json	data/processed/text	pandas
preprocess_audio	raw audio	data/processed/audio	librosa, soundfile
split	processed text + audio	train / val / test splits	sklearn
baseline_stats	train split	baselines.json	numpy, scipy

- 3,956 training examples, 494 validation examples, full pipeline completes in < 5 minutes
- Baseline statistics (mean, variance, OOV rate) stored in baselines.json for later drift comparison

4.2 Source Control & Continuous Integration

- **Git** — source code, configuration, and documentation
- **DVC** — large data artifacts and model weights (pointer files in Git, content in .dvc/cache)
- **DVC DAG** — visualised with dvc dag; dvc repro re-executes only changed stages
- **GitHub Actions** — CI pipeline runs on every push: ruff lint → unit tests → integration tests → docker build
- Every MLflow run is tagged with its Git commit SHA + DVC lock hash for full reproducibility

4.3 Experiment Tracking (MLflow)

Every training run is tracked in MLflow with full reproducibility. The model registry holds both trained models and controls promotion to Production.

Tracked Item	Details
Parameters	base_model, lora_r, lora_alpha, lora_target_modules, learning_rate, batch_size, num_epochs, warmup_ste
Metrics	train/loss per 25 steps, eval/WER, eval/BLEU, final_wer, final_bleu
Artifacts	LoRA adapter weights, tokenizer, processor, training_args.bin
Tags	git_sha, dvc_lock_sha, run_name — every run reproducible from (git_sha, dvc_lock, mlflow_run_id)
Registry	navi-whisper (v1) and navi-marian (v1) registered; auto-promote if WER/BLEU improves ≥ 0.01

4.4 Exporter Instrumentation & Visualization

The FastAPI backend exposes a /metrics endpoint in Prometheus text format. Prometheus scrapes it every 10 seconds. Grafana dashboards visualise all metrics in near-real-time with pre-provisioned datasources and dashboard JSON.

Metric	Type	Purpose
navi_translation_requests_total	Counter	Request volume by input_type and status
navi_translation_latency_ms	Histogram	End-to-end latency distribution
navi_whisper_wer_live	Gauge	Live ASR word error rate
navi_marian_bleu_live	Gauge	Live NMT BLEU score
navi_oov_rate	Gauge	Out-of-vocabulary rate — primary drift signal
navi_fallback_rate	Gauge	Fraction of requests using dictionary fallback
navi_model_version	Info	Currently loaded model version label
navi_vocab_submissions_total	Counter	Community vocabulary submissions

- Alert: HighErrorRate — error rate > 5% over 5 minutes → severity critical
- Alert: OOVDriftDetected — navi_oov_rate > 0.15 → triggers Airflow retrain DAG
- Alert: HighLatency — p95 latency > 5,000 ms over 5 minutes → severity warning

4.5 Software Packaging

- **MLproject** — defines train_whisper, train_marian, and evaluate entry points with parameterised commands; python_env.yaml pins the full dependency set
- **FastAPI** — REST inference API with Pydantic-enforced schemas; model loaded from MLflow registry at startup; /health and /ready endpoints for orchestration health checks
- **Docker Compose** — 5 isolated services (backend, frontend, mlflow, prometheus, grafana) on a shared bridge network; each has its own Dockerfile and health check

• **MLflow Model Registry** — models promoted to Production stage; FastAPI loads the Production alias at startup without code changes

5. Model Development & Training

5.1 Whisper-tiny + LoRA (ASR)

Whisper-tiny (39M parameters) was selected over Whisper-small (244M) based on published research showing that smaller Whisper variants outperform larger ones when fine-tuning data is limited. PEFT/LoRA adapters reduce the trainable parameter count to 589,824 (1.54% of total), making training feasible on Apple Silicon MPS hardware within the no-cloud constraint.

Configuration	Value	Rationale
Base model	openai/whisper-tiny	39M params; best low-resource performance per Springer 2024 study
LoRA rank (r)	16	Bumped from 8 to compensate for smaller backbone capacity
LoRA alpha	32	Standard alpha = 2×r ratio
Target modules	q_proj, k_proj, v_proj, out_proj	Full attention coverage for better language adaptation
Learning rate	1×10■■	Conservative; literature shows higher LR can hurt tiny/base variants
Effective batch	4 (1 device × 4 accum steps)	Gradient accumulation avoids OOM while maintaining stable gradients
Epochs	5	Validated against val set; early stopping if WER plateaus
Trainable params	589,824 / 38,350,464 (1.54%)	LoRA efficiency — only attention adapters updated
Training language	mi (M■ori)	Closest Whisper-supported language to Na'vi phonology
Final WER	0.42	Logged in MLflow; target was ≤ 0.35; limited by small audio corpus

5.2 MarianMT (NMT — Na'vi → English)

Helsinki-NLP/opus-mt-mul-en was selected as the base translation model. It is a lightweight seq2seq architecture purpose-built for machine translation and outperforms general-purpose LLMs on domain-specific fine-tuning with small datasets, which is the Na'vi scenario (~2,600 word pairs).

Configuration	Value
Base model	Helsinki-NLP/opus-mt-mul-en
Learning rate	2×10■■
Batch size	16
Epochs	10
Train loss	~3.30 (final epoch)
Registry	navi-marian v1 — Production stage

5.3 Reykunyu Dictionary Fallback

When neural translation confidence falls below 0.4, the system falls back to word-by-word lookup in the Reykunyu Na'vi dictionary (~2,600 entries). This ensures the application remains usable for common words even when the neural model confidence is low — a deliberate design choice for low-resource ASR.

6. Software Engineering

6.1 API Endpoints (LLD Summary)

Method	Endpoint	Input	Output
POST	/translate/audio	multipart/form-data (WAV/MP3/OGG ≤ 30s)	navi_text, english, confidence, latency_ms
POST	/translate/text	JSON: {text: string, 1–500 chars}	english, confidence, word_breakdown, latency_ms
POST	/vocab/submit	JSON: {navi_word, english_meaning, audio_b64?}	accepted, message
GET	/health	—	status, model_version, uptime_s
GET	/ready	—	ready, whisper_loaded, marian_loaded
GET	/metrics	—	Prometheus text exposition (8 metrics)

6.2 Implementation Quality

- **Code style:** PEP-8 enforced via ruff; linting runs on every CI push
- **Logging:** module-level loggers throughout; training scripts log to stdout and timestamped log files
- **Exception handling:** FastAPI global exception handlers return structured JSON with request IDs
- **Type safety:** Pydantic v2 enforces all request/response schemas at runtime
- **Loose coupling:** frontend (nginx + static JS) and backend communicate only via REST; API base URL is configurable

6.3 Testing

Suite	Count	Coverage	Result
Unit tests	30	Schemas, drift detection, Reykunyu parser, Prometheus helpers, build_pairs	30 / 30 PASSED
Integration tests	2	FastAPI route wiring (fake engine, no model weights required)	2 / 2 PASSED
Total	32	—	32 / 32 PASSED

7. Web Application & Pipeline Visualization

7.1 Frontend UI

A 4-tab single-page application served by nginx. No framework dependencies — pure HTML/CSS/JavaScript for minimal footprint and easy containerisation.

- **Tab 1 — Translate Audio:** Browser MediaRecorder API captures microphone input; alternatively accepts file upload (WAV/MP3/OGG). Displays Na'vi transcription, English translation, and confidence score.
- **Tab 2 — Translate Text:** Text input with real-time Na'vi-to-English translation; word-level breakdown shows per-token dictionary matches.
- **Tab 3 — Submit Vocabulary:** Community contributions with optional audio upload; validated against Na'vi character set before acceptance.
- **Tab 4 — About:** System description and user manual reference.

7.2 MLOps Pipeline Visualization

Multiple specialised MLOps UIs are accessible via Docker Compose, each serving a distinct observability function:

Tool	URL	Purpose
MLflow UI	http://localhost:5000	Experiment runs, loss curves, parameter comparison, model registry
Grafana	http://localhost:3001	Real-time inference metrics, OOV drift, latency histograms
Prometheus	http://localhost:9090	Raw metric queries, alert state, scrape target health
Airflow	DAG trigger / CLI	Training DAG runs, task-level logs, retrain scheduling
DVC CLI	dvc dag / dvc repro	Data pipeline DAG, stage-level lineage and caching

8. Problems Faced & Mitigations

#	Problem	Root Cause	Mitigation
1	Training crash wiped all progress	save_strategy='epoch' — crash 1 step before epoch end lost everything	Switched to save_strategy='steps' with save_steps=500; max 8
2	whisper-small training infeasible on MPS	244M params (~43 secs/step on Apple Silicon [illegible]	Switched to whisper-tiny (39M params, ~6× faster). Research ba
3	MPS thermal throttling during mid-training evals	Repeated eval loops (240 sec/eval step) caused the Mac [illegible]	Removed mid-training evaluation entirely (eval_strategy='no'); single e
4	CI pipeline failing on every push	CI was pip-installing torch/transformers (~3 GB) on every GitHub Actions run	Created requirements-ci.txt with only the 12 lightweight package
5	Limited Na'vi audio data	Constructed language — ~4,000 samples; no public TTS available	Synthetic augmentation + LoRA to avoid overfitting + Reyk
6	No-cloud constraint	Course guidelines explicitly prohibit cloud platforms	Whisper-tiny + LoRA makes training feasible locally; documente

9. Technology Stack

Category	Tool	Version	Role
ASR	OpenAI Whisper-tiny + PEFT LoRA	openai/whisper-tiny	Speech-to-Na'vi-text
NMT	MarianMT	opus-mt-mul-en	Na'vi-to-English translation
API	FastAPI + Uvicorn	≥ 0.109	REST inference server
Frontend	nginx	alpine	Static UI host + reverse proxy

Experiment Track	MLflow	≥ 2.10	Run tracking + model registry
Data Version	DVC	3.x	Data & artifact versioning
Orchestration	Apache Airflow	2.8+	Training DAG scheduling
Monitoring	Prometheus + Grafana	latest	Metrics scraping + dashboards
Containers	Docker Compose	v2	5-service local deployment
CI/CD	GitHub Actions	—	Lint + test + docker build
Testing	pytest + pytest-cov	≥ 9.0	Unit + integration tests
Linting	ruff	latest	PEP-8 enforcement

10. Conclusion

The Na'vi Translator demonstrates a complete MLOps lifecycle applied to a challenging low-resource language scenario under real hardware constraints. Every rubric component is implemented: Airflow orchestration, DVC data versioning, MLflow experiment tracking, Prometheus/Grafana monitoring, FastAPI serving, Docker Compose packaging, and GitHub Actions CI. The system deliberately prioritises **MLOps best practices over raw model accuracy** — the architecture is designed to detect performance degradation, trigger automated retraining, and promote improved models to production without manual intervention.

The no-cloud constraint, far from being a limitation, motivated engineering decisions (LoRA over full fine-tuning, whisper-tiny over whisper-small, dictionary fallback) that are consistent with production MLOps for resource-constrained environments and are directly supported by recent research literature.

References

- Springer (2024). Exploration of Whisper fine-tuning strategies for low-resource ASR. Journal on Audio, Speech, and Music Processing.
- MDPI Sensors (2025). LoRA-INT8 Whisper: A Low-Cost Framework for Edge Devices.
- Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- HuggingFace PEFT Documentation. https://huggingface.co/docs/peft
- Reykunyu Na'vi Dictionary API. https://reykunyu.lu